

AI Helpy Chat QA Automation

Troubleshooting Report - 2026년 6월 22일 작업 정리

항목	내용
프로젝트	AI Helpy Chat Quiz / PPT / Deep 생성 기능 QA 자동화
주요 기술	Python 3.14, pytest, pytest-xdist, Selenium, Docker Compose, Jenkins, Allure, Slack, Jira
CI 실행 방식	Jenkins는 유지하고, 테스트 실행 환경만 Docker Compose로 분리
테스트 범위	상세 테스트 30건 기준: Quiz, PPT, Deep 상세 검증 / E2E slow 파일은 기본 CI 제외
최종 상태	README 작성 및 push 완료. Allure Report 생성 확인. Deep 상세 테스트 2건 Timeout 이력 확인

1. 전체 작업 요약

오늘 작업은 단순한 테스트 코드 수정이 아니라, QA 자동화 프로젝트를 포트폴리오형 CI 파이프라인으로 정리하는 흐름이었다. 테스트 구조를 먼저 정리하고, Jenkins 실행 방식을 Docker 기반으로 바꾼 뒤, JUnit과 Allure 결과를 Jenkins에서 확인할 수 있도록 연결했다.

- Quiz, PPT, Deep 테스트를 상세 테스트와 slow E2E 테스트로 분리
- 공통 상수와 helper를 tests/*_base.py로 분리
- Jenkinsfile을 Docker Compose 기반 실행 구조로 변경
- Selenium Chrome 컨테이너와 tests 컨테이너를 분리
- Allure 원본 결과 생성 및 Jenkins Allure Report publish 연결
- README를 포트폴리오형 QA 자동화 문서로 작성하고 GitHub main 브랜치에 push

2. 테스트 파일 분리 및 base 모듈 정리

목적

기존 test_quiz_create.py, test_ppt_create.py, test_deep_create.py에 상세 검증과 E2E 테스트가 섞여 있어 관리가 어려웠다. 기본 CI에서는 빠른 상세 테스트만 돌리고, slow E2E는 필요할 때 별도로 실행할 수 있는 구조가 필요했다.

작업 내용

- 상세 테스트 파일은 test_quiz_create.py, test_ppt_create.py, test_deep_create.py로 유지
- slow E2E 테스트는 test_quiz_create_E2E.py, test_ppt_create_E2E.py, test_deep_create_E2E.py로 분리
- quiz_create_base.py, ppt_create_base.py, deep_create_base.py에 공통 상수와 helper를 분리
- 긴 from import 목록 대신 import *_create_base as base 형태로 가독성 개선

문제 / 증상

초기에는 공통 helper를 pages 레이어로 옮길지, tests 전용 base 모듈로 둘지 기준이 필요했다. pages는 실제 페이지 조작 책임이고, 테스트 데이터와 검증 helper는 테스트 책임이라 경계가 섞일 위험이 있었다.

원인 분석

Page Object는 제품 UI 조작을 표현하는 계층이고, 테스트 상수/검증 helper는 테스트 시나리오를 표현하는 계층이다. 두 책임을 섞으면 pages가 테스트 전용 로직에 오염될 수 있다.

해결 / 조치

- 테스트 전용 base 모듈을 tests 하위에 배치
- pages/quiz_page.py 등 Page Object 파일은 건드리지 않음
- slow 테스트는 E2E 파일에만 남기고 상세 테스트 파일에서는 제외

결과

테스트 구조가 상세 테스트와 E2E 테스트로 나뉘어 Jenkins 기본 CI에서 E2E를 제외하기 쉬워졌다. 유지보수 기준도 pages와 tests로 분리되어 명확해졌다.

3. Jenkins Pipeline 구성

목적

GitHub에 push하면 Jenkins에서 자동으로 테스트가 실행되고, 결과를 JUnit/Allure/Artifact로 확인할 수 있는 CI 흐름을 만드는 것이 목표였다.

작업 내용

- Jenkinsfile 추가 및 Declarative Pipeline 구성
- Jenkins job을 Pipeline script from SCM 방식으로 설정
- GitHub push trigger와 Jenkins job 실행 흐름 확인
- Credentials로 TEST_USER_PW, JIRA_API_TOKEN, Slack Webhook을 주입

문제 / 증상

초기 Jenkins 실행은 0.73초 수준으로 끝나며 실제 pytest가 실행되지 않았다. Jenkins job에 저장소 Jenkinsfile이 연결되지 않았거나, Pipeline script가 비어 있는 상태였다.

원인 분석

Jenkins job이 실제 GitHub 저장소의 Jenkinsfile을 읽지 않으면 파이프라인 시작/종료만 찍히고 테스트 stage가 실행되지 않는다.

해결 / 조치

- Jenkins job 설정에서 Pipeline script from SCM을 선택
- Repository URL, Credentials, Branch, Script Path=Jenkinsfile 기준으로 설정
- Jenkins 콘솔에서 pytest 실행 로그와 xdist worker 로그를 확인

결과

Jenkins에서 실제 테스트가 실행되기 시작했고, 빌드 페이지에서 테스트 결과와 Allure Report 링크를 확인할 수 있게 되었다.

4. Windows Jenkins 콘솔 한글 깨짐 해결

목적

Jenkins 콘솔에서 pytest live log의 한글 메시지가 깨져 보이는 문제를 줄이고, Python/Java/Windows cmd 인코딩을 UTF-8로 맞추는 것이 목표였다.

작업 내용

- Jenkinsfile environment에 JAVA_TOOL_OPTIONS='-Dfile.encoding=UTF-8' 추가
- bat 블록마다 chcp 65001 실행
- Dockerfile에 PYTHONUTF8=1, PYTHONIOENCODING=utf-8 설정

문제 / 증상

콘솔에 '테스트 세션 시작' 같은 문구가 깨져 출력되었다.

원인 분석

Windows 로컬 Jenkins는 cmd 코드페이지, JVM 파일 인코딩, Python 표준 입출력 인코딩이 서로 다르면 UTF-8 로그를 올바르게 표시하지 못한다.

해결 / 조치

- JVM, Windows cmd, Python 환경변수를 모두 UTF-8 방향으로 맞춤
- Jenkinsfile 변경 후 commit/push하여 Jenkins가 최신 Jenkinsfile을 사용하게 함

결과

이후 Docker 컨테이너 내부 pytest 로그에서는 한글 출력이 정상적으로 보이는 흐름을 확인했다. 다만 Jenkins UI/플러그인/브라우저 렌더링 상태에 따라 일부 과거 로그는 그대로 깨져 보일 수 있다.

5. Docker Compose 기반 테스트 실행 환경 분리

목적

로컬 Windows/Jenkins 환경 차이에 덜 흔들리도록 테스트 실행 환경을 컨테이너로 고정하는 것이 목표였다. Jenkins는 오케스트레이션만 담당하고, 실제 pytest와 Chrome은 Docker에서 실행하도록 분리했다.

작업 내용

- Dockerfile 추가: Python 3.14 기반 테스트 이미지 생성
- docker-compose.yml 추가: selenium/standalone-chrome 서비스와 tests 서비스 분리
- SELENIUM_REMOTE_URL=http://selenium:4444/wd/hub 주입
- reports, logs, allure-results를 volume으로 Jenkins workspace에 남김
- pytest-xdist -n 3으로 3 workers 병렬 실행

문제 / 증상

Docker Desktop Images 화면만 보고 실행 여부를 판단하기 어려웠다. 또한 tests 컨테이너가 Stopped로 표시되어 실패인지 완료인지 혼동이 있었다.

원인 분석

Docker 이미지와 컨테이너는 다르다. Images는 빌드된 실행 템플릿이고, Containers는 실제 실행 이력이다. pytest 컨테이너는 테스트가 끝나면 정상/실패와 무관하게 멈추는 것이 자연스럽다.

해결 / 조치

- 실행 확인 기준을 Docker Desktop Containers 탭과 터미널 로그로 구분
- 터미널에서 'Container ... Healthy', '3 workers [30 items]', PASSED 로그를 확인
- Jenkins에서는 --exit-code-from tests로 tests 컨테이너의 종료 코드를 빌드 결과에 반영

결과

Docker 기반 실행이 확인되었고, Jenkins도 같은 compose 명령으로 테스트를 실행하도록 통일되었다.

6. Allure Report 연동

목적

JUnit XML만으로는 테스트 흐름과 카테고리 확인이 부족하므로, Jenkins에서 Allure Report를 함께 볼 수 있도록 구성하는 것이 목표였다.

작업 내용

- requirements.txt에 allure-pytest 추가
- docker-compose.yml pytest command에 --alluredir=allure-results, --clean-alluredir 추가
- Jenkins Allure Plugin 설치 및 Tools에서 Allure Commandline 설정
- Jenkinsfile post 단계에 allure(results: [[path: 'allure-results']]) 추가
- 테스트 클래스/함수에 @allure.feature, @allure.story, @allure.title 메타데이터 추가

문제 / 증상

Allure Report 페이지는 열렸지만, 일부 위젯에 500 메시지 또는 JSON/HTML 파싱 오류처럼 보이는 영역이 표시되었다. 또한 코드에 with allure.step을 많이 넣으면 테스트가 지저분해질 우려가 있었다.

원인 분석

Allure는 원본 결과 파일이 생성되고 Jenkins plugin이 이를 HTML로 publish해야 한다. 초기 단계에서는 step을 무리하게 넣기보다 feature/story/title만 붙여도 충분히 분류성이 좋아진다.

해결 / 조치

- 테스트 내부 step 남발은 보류
- feature/story/title 중심으로 최소 메타데이터 적용
- allure-results를 artifact로도 보관하여 원본 결과 확인 가능하게 구성

결과

Jenkins 빌드 페이지에서 Allure Report 메뉴가 생겼고, 30개 테스트 케이스와 결과 요약을 확인했다.

7. Jenkins #11 실패 분석

목적

Jenkins 빌드가 빨간 X로 표시된 원인이 Allure/Jenkins 설정 문제인지, 실제 테스트 실패인지 구분하는 것이 목표였다.

작업 내용

- Jenkins 빌드 #11 테스트 결과 확인
- JUnit XML 기준 tests=30, failures=2, skipped=4 확인
- 실패 테스트 이름 확인: Deep Investigation 상세 테스트 2건

문제 / 증상

Jenkins 메인 화면에 빨간 X가 표시되고, Allure trend에서도 실패가 표시되었다. 사용자는 xfail 4건 때문에 실패로 보이는지 의심했다.

원인 분석

xfail은 pytest/JUnit에서 skipped 또는 expected failure 성격으로 기록되며 빌드 실패 원인이 아니다. #11의 빨간 X는 Deep 상세 테스트 2건이 TimeoutException으로 실제 실패했기 때문에 발생했다.

해결 / 조치

- 실패 2건: test_instructions_validation[chrome], test_regenerate_button_state[chrome]
- xfail 4건은 expected failure로 유지되는 항목으로 분류
- 현재 이슈를 README Known Issues / Current Notes에 보수적으로 기록

결과

빌드 실패 원인은 Jenkins/Allure 자체가 아니라 Docker/Selenium 환경에서 Deep 상세 테스트 2건이 timeout 난 것이다. 다음 개선 대상은 Deep 페이지 wait 조건, selector 안정성, Docker Selenium 응답 시간 보정이다.

8. GitHub README 작성 및 push

목적

프로젝트를 포트폴리오로 설명할 수 있도록 실제 구현된 범위만 README에 정리하는 것이 목표였다.

작업 내용

- README.md 신규 작성
- Overview, Key Achievements, Tech Stack, Test Scope, Project Structure, Run Tests, CI Pipeline, Allure, Slack/Jira, Known Issues 포함
- 민감정보 검색 후 README에 토큰/웹훅/비밀번호가 없는지 확인
- git commit -m 'Add project README' 후 main 브랜치로 push

문제 / 증상

README에 실제 구현되지 않은 기능까지 적으면 포트폴리오 신뢰도가 떨어질 수 있었다. 또한 .env에 민감정보가 있어 문서나 커밋에 노출되지 않도록 확인이 필요했다.

원인 분석

README는 구현 사실과 실행 방법 중심으로 작성해야 한다. GitHub Actions, Allure screenshot attachment, 세부 allure.step 등 아직 구현하지 않은 항목은 완료 기능처럼 쓰면 안 된다.

해결 / 조치

- 실제 구현된 Selenium, pytest, Docker Compose, Jenkins, Allure, Slack, Jira, xfail만 포함
- 민감정보는 변수명만 설명하고 값은 제외
- 최종 커밋 02c40a6 Add project README로 push 완료

결과

GitHub 저장소에 README가 반영되었고, 이후 수정 방향을 잡을 수 있는 1차 포트폴리오 문서가 준비되었다.

9. 현재 남은 이슈와 다음 액션

현재 CI/CD 빠대는 동작한다. 남은 핵심 이슈는 Jenkins #11에서 확인된 Deep 상세 테스트 2건 TimeoutException이다. 이 문제는 Allure나 Docker 설정 문제가 아니라 테스트 안정성 문제로 보는 것이 타당하다.

- Deep 테스트 실패 2건의 selector/wait 조건 재검토
- Docker Selenium 환경에서 페이지 로딩 및 생성 버튼 상태 변화 대기 시간 보정
- 실패 시 screenshot/page source를 Allure attachment로 남기도록 개선
- Allure categories/environment 파일 추가로 리포트 가독성 개선
- README에 테스트 결과 스크린샷 또는 Jenkins/Allure 캡처 추가 여부 검토

10. 실행 명령 정리

Docker 기반 상세 테스트 실행

```
docker compose up --build --abort-on-container-exit --exit-code-from tests tests
```

로컬 pytest 상세 테스트 실행

```
pytest tests/test_quiz_create.py tests/test_ppt_create.py tests/test_deep_create.py -n 3 --browser chrome
```

slow E2E 별도 실행

```
pytest tests/test_quiz_create_E2E.py tests/test_ppt_create_E2E.py tests/test_deep_create_E2E.py -m slow --browser chrome
```

11. 최종 판단

오늘 작업은 테스트 코드 정리, CI 실행 환경 분리, 리포트 체계 구축까지 이어진 자동화 프로젝트 고도화 작업이다. Jenkins와 Docker Compose, Allure가 연결되었고, GitHub README까지 정리되어 포트폴리오로 설명 가능한 형태가 되었다.

다만 자동화 프로젝트의 품질은 실패가 없게 숨기는 것이 아니라 실패 원인을 추적 가능하게 만드는 것에서 나온다. 현재 Deep 2건 Timeout은 다음 리팩토링/안정화 대상이며, Allure와 JUnit으로 실패 위치가 드러나도록 만든 점은 이번 작업의 중요한 성과다.